

## Phase 5: Apex programming (SmartShop)

While most logic runs in Flows, Phase 5 adds a minimal, scalable Apex layer where server-side rollups and robustness are needed, using one-trigger-per-object and a logic-less trigger + handler pattern for maintainability.

All queries/DML are bulkified, and totals use the native `OrderItem.TotalPrice` field, avoiding arithmetic expressions in SOQL and staying aligned with standard order math.

### Phase 5.1.1: OrderItemTotals trigger

- Object: `OrderItem`.
- Trigger context: after insert, after update, after delete, after undelete, so the Order total stays correct whenever lines change.
- Why a thin trigger: one per object avoids execution-order issues and routes all logic to a handler class with clear context methods.

Execution condition and logic

- The handler collects impacted Order Ids from `Trigger.new/old`, runs a single aggregate query `SELECT OrderId, SUM(TotalPrice) GROUP BY OrderId`, and updates only those Orders.
- Using `TotalPrice` ensures parity with platform calculations and prevents invalid aggregate expressions in SOQL.

Trigger (logic-less)

text

```
trigger OrderItemTrigger on OrderItem (  
    after insert, after update, after delete, after undelete  
) {  
    OrderItemTriggerHandler.run();  
}
```

This trigger delegates all work to a handler, keeping the orchestration centralized and testable.

Handler (bulk-safe `SUM(TotalPrice)`)

text

```
public class OrderItemTriggerHandler {  
    private static Boolean isRunning = false;
```

```

public static void run(){
    if (isRunning) return;
    isRunning = true;
    try {
        Set<Id> orderIds = new Set<Id>();
        if (Trigger.isDelete) {
            for (OrderItem oi : Trigger.old) if
(oi.OrderId != null) orderIds.add(oi.OrderId);
        } else {
            for (OrderItem oi : Trigger.new) if
(oi.OrderId != null) orderIds.add(oi.OrderId);
        }
        if (!orderIds.isEmpty()) recalcOrders(orderIds);
    } finally {
        isRunning = false;
    }
}

```

```

public static void recalcOrders(Set<Id> orderIds){
    Map<Id, Decimal> totals = new Map<Id, Decimal>();
    for (AggregateResult ar : [
        SELECT OrderId oid, SUM(TotalPrice) tot
        FROM OrderItem
        WHERE OrderId IN :orderIds
        GROUP BY OrderId
    ]) {
        totals.put((Id) ar.get('oid'), (Decimal)
ar.get('tot'));
    }
    List<Order> ups = new List<Order>();
    for (Id oid : orderIds) {
        Decimal t = totals.containsKey(oid) ?
totals.get(oid) : 0;
        ups.add(new Order(Id = oid, Line_Net_Total__c =
t));
    }
    if (!ups.isEmpty()) update ups;
}

```

```
}
```

The handler guards recursion, aggregates once per batch, and performs a single bulk update to remain within governor limits.

## Phase 5.1.2: Unit testing (coverage + assertions)

- Approach: use @testSetup to seed Accounts, Products, Standard Pricebook, and a Draft Order; then assert totals after insert, update, and delete of OrderItems.
- Use Test.startTest()/Test.stopTest() to reset governor limits and finalize async work if any, ensuring deterministic results.

Test class

```
text
```

```
@IsTest
```

```
private class OrderItemApexTest {
```

```
    @testSetup
```

```
    static void data() {
```

```
        Account a = new Account(Name='ABC Electronics');
```

```
insert a;
```

```
        Pricebook2 std = [SELECT Id FROM Pricebook2 WHERE  
IsStandard=true LIMIT 1];
```

```
        Product2 p = new Product2(Name='LED Desk Lamp',  
IsActive=true); insert p;
```

```
        insert new PricebookEntry(Pricebook2Id=std.Id,  
Product2Id=p.Id, UnitPrice=1999, IsActive=true);
```

```
        Order o = new Order(AccountId=a.Id,  
EffectiveDate=Date.today(), Status='Draft',  
Pricebook2Id=std.Id);
```

```
        insert o;
```

```
    }
```

```
@IsTest
```

```
    static void rollup_insert_update_delete() {
```

```
        Order o = [SELECT Id, Line_Net_Total__c FROM Order  
LIMIT 1];
```

```
        PricebookEntry pbe = [SELECT Id, UnitPrice FROM
PricebookEntry LIMIT 1];
```

```
        Test.startTest();
        OrderItem oi1 = new OrderItem(OrderId=o.Id,
PricebookEntryId=pbe.Id, Quantity=2, UnitPrice=1999);
        OrderItem oi2 = new OrderItem(OrderId=o.Id,
PricebookEntryId=pbe.Id, Quantity=3, UnitPrice=1999);
        insert new List<OrderItem>{ oi1, oi2 };
        Test.stopTest();
```

```
        Order afterInsert = [SELECT Line_Net_Total__c FROM
Order WHERE Id = :o.Id];
        System.assertEquals(5 * 1999,
afterInsert.Line_Net_Total__c);
```

```
        Test.startTest();
        oi1 = [SELECT Id, Quantity FROM OrderItem WHERE Id =
:oi1.Id];
        oi1.Quantity = 4;
        update oi1;
        Test.stopTest();
```

```
        Order afterUpdate = [SELECT Line_Net_Total__c FROM
Order WHERE Id = :o.Id];
        System.assertEquals((4 + 3) * 1999,
afterUpdate.Line_Net_Total__c);
```

```
        Test.startTest();
        delete oi2;
        Test.stopTest();
```

```
        Order afterDelete = [SELECT Line_Net_Total__c FROM
Order WHERE Id = :o.Id];
        System.assertEquals(4 * 1999,
afterDelete.Line_Net_Total__c);
```

```
    }
}
```

This test demonstrates predictable assertions across insert/update/delete and leverages startTest/stopTest per best practices.